

메아리의 숲 (Echo Forest) - 포팅 매뉴얼

프로젝트 버전: v1.0.1

작성일: 2026-02-07

목차

1. 시스템 아키텍처
2. 서버 사양 및 요구사항
3. 환경 변수 설정
4. Docker 이미지 빌드
5. 서비스 배포 - 서버 (Docker Compose) [권장]
6. 로컬 개발 환경 실행
7. 외부 서비스 정보
8. DB 덤프 파일
9. 시연 시나리오

1. 시스템 아키텍처

본 프로젝트는 Docker Compose를 사용하여 모든 서비스를 컨테이너로 운영합니다.

[서비스 구성]

서비스명	이미지	포트	설명
game-server	echoforest-server:latest	9001 (내부)	Spring Boot 게임 서버
ai-server	echoforest-ai:latest	8000 (내부)	FastAPI 감정 분석 AI
mysql	mysql:8.0	3307:3306	데이터베이스
redis	redis:alpine	6379	세션/캐시 저장소
livekit	livekit/livekit-server:latest	60000-60200	실시간 음성/영상 통신
nginx	nginx:latest	80, 443	웹 서버 (리버스 프록시)
jenkins	jenkins/jenkins:its	(내부)	CI/CD 자동 배포
prometheus	prom/prometheus:latest	9090	메트릭 수집

grafana	grafana/grafana:latest	3000	모니터링 대시보드
---------	------------------------	------	-----------

[네트워크 구성]

- 모든 서비스는 'echo-net' 브리지 네트워크로 연결
- 서비스 간 통신은 컨테이너 이름으로 접근 (예: mysql, redis, ai-server)

[JVM / WAS / 웹서버 정보]

구분	제품명	버전	설정값/비고
JVM	BellSoft Liberica OpenJDK	21	Docker 이미지: bellsoft/liberica-openjdk-alpine:21
WAS	Spring Boot (내장 Tomcat)	3.5.9	server.port=9001, Virtual Threads 활성화
웹서버	Nginx	latest (Docker)	리버스 프록시, SSL 종단, 정적 파일 서빙
빌드 도구	Gradle	8.5	Docker 이미지: gradle:8.5-jdk21

[프론트엔드 정보]

구분	제품명	버전
런타임	Node.js	20.x LTS
패키지 관리	npm	10.x
빌드 도구	Vite	7.x
프레임워크	React	19.x

[AI 서버 정보]

구분	제품명	버전
런타임	Python	3.12
웹 프레임워크	FastAPI	0.100+
ASGI 서버	Uvicorn	0.23+
ML 프레임워크	PyTorch	2.0+

=====

2. 서버 사양 및 요구사항

=====

2.1 서버 사양

항목	권장 사양

OS	Ubuntu 20.04 LTS 이상
CPU	4코어 이상
RAM	16GB 이상 (AI 서버 4GB 할당)
디스크	100GB SSD 이상
네트워크	공인 IP 필요, 방화벽 포트 개방 필요

2.2 필수 설치 소프트웨어

소프트웨어	버전	설치 명령어

Docker	24.x	sudo apt install docker.io
Docker Compose	2.x	sudo apt install docker-compose-plugin
Git	최신	sudo apt install git

2.3 방화벽 포트 개방

포트	프로토콜	용도

80	TCP	HTTP (Nginx)
443	TCP	HTTPS (Nginx + SSL)
3307	TCP	MySQL (외부 접속 시)
6379	TCP	Redis (외부 접속 시)
9090	TCP	Prometheus (모니터링)
3000	TCP	Grafana (모니터링)
3478	TCP/UDP	TURN 서버 (LiveKit)
60000-60200	UDP	WebRTC 미디어 포트 (LiveKit)

3. 환경 변수 설정

3.1 백엔드 환경 변수

파일 경로: echoforest-backend/.env

```
-----
# =====
```

백엔드 환경 변수 (.gitignore 필수)

=====

LiveKit 설정

LIVEKIT_URL=ws://echoforest-livekit:7880

LIVEKIT_API_KEY={LIVEKIT_API_KEY}

LIVEKIT_API_SECRET={LIVEKIT_API_SECRET}

AI 모델 설정 (SSAFY GMS API)

AI_MODEL_URL=https://gms.ssafy.io/gmsapi/api.openai.com/v1/images/generations

AI_MODEL_API_KEY={AI_MODEL_API_KEY}

DB 설정 (docker-compose의 mysql 서비스명 사용)

DB_URL=jdbc:mysql://mysql:3306/echoforest?useSSL=false&allowPublicKeyRetrieval=true&characterEncoding=UTF-8&serverTimezone=Asia/Seoul

DB_USERNAME={DB_USERNAME}

DB_PASSWORD={DB_PASSWORD}

DB_ROOT_PASSWORD={DB_ROOT_PASSWORD}

JWT

JWT_SECRET_KEY={JWT_SECRET_KEY}

JWT_ACCESS_TOKEN_VALIDITY=86400000

Gmail SMTP

MAIL_USERNAME={MAIL_USERNAME}

MAIL_PASSWORD={MAIL_PASSWORD}

3.2 프론트엔드 환경 변수

파일 경로: echoforest-frontend/.env

=====

프론트엔드 환경 변수

=====

VITE_API_BASE_URL=https://i14d105.p.ssafy.io/api

VITE_LIVEKIT_URL=wss://i14d105.p.ssafy.io/livekit

[주요 계정 및 비밀번호 정리]

항목	값
MySQL root 비밀번호	{DB_ROOT_PASSWORD}
MySQL 사용자 계정	{DB_USERNAME}
MySQL 사용자 비밀번호	{DB_PASSWORD}
Gmail 계정	{MAIL_USERNAME}
Gmail 앱 비밀번호	{MAIL_PASSWORD}
Jenkins 초기 비밀번호	docker exec echoforest-jenkins cat /var/jenkins_home/secrets/initialAdminPassword

4. Docker 이미지 빌드

4.1 백엔드 서버 이미지 빌드

```
cd echoforest-backend
docker build -t echoforest-server:latest .
```

빌드 시간: 약 3-5분 (첫 빌드 시 Gradle 의존성 다운로드)

4.2 AI 서버 이미지 빌드

```
cd echoforest-ai/inference
docker build -t echoforest-ai:latest .
```

빌드 시간: 약 5-10분 (PyTorch, Transformers 등 대용량 라이브러리 설치)

4.3 프론트엔드 빌드

프론트엔드는 Jenkins에서 자동 빌드되거나, 수동으로 빌드 후 Nginx가 서빙합니다.

```
cd echoforest-frontend
npm install
npm run build
```

빌드 결과물: dist/ 폴더 (이 폴더를 Nginx가 서빙)

=====

5. 서비스 배포 - 서버 (Docker Compose) [권장]

=====

5.1 전체 서비스 시작

```
cd echoforest-backend
docker compose up -d
```

5.2 서비스 상태 확인

```
docker compose ps
```

정상 상태 예시:

NAME	STATUS
echoforest-server	Up
echoforest-ai	Up
echoforest-mysql	Up
echoforest-redis	Up
echoforest-livekit	Up
echoforest-nginx	Up
echoforest-jenkins	Up
echoforest-prometheus	Up
echoforest-grafana	Up

5.3 로그 확인

전체 로그

```
docker compose logs -f
```

특정 서비스 로그

```
docker compose logs -f game-server
```

```
docker compose logs -f ai-server
```

5.4 서비스 재시작

```
# 전체 재시작
docker compose restart

# 특정 서비스만 재시작
docker compose restart game-server
```

5.5 서비스 중지 및 삭제

```
# 서비스 중지 (데이터 보존)
docker compose down

# 서비스 중지 + 볼륨 삭제 (데이터 삭제됨, 주의!)
docker compose down -v
```

6. 로컬 개발 환경 실행

로컬에서 개발 및 테스트를 위해 Docker를 사용하여 백엔드를 실행할 수 있습니다.

[사전 요구사항]

- Docker Desktop 설치 (<https://www.docker.com/products/docker-desktop>)
- Git으로 프로젝트 클론

6.1 환경 변수 설정

1. .env.template 파일을 .env로 복사:

```
cd echoforest-backend
cp .env.template .env
```

2. .env 파일을 열어 실제 값으로 수정:

- DB_PASSWORD, DB_ROOT_PASSWORD: 원하는 비밀번호 설정
- JWT_SECRET_KEY: 임의의 문자열
- AI_MODEL_API_KEY: SSAFY GMS API 키
- MAIL_USERNAME, MAIL_PASSWORD: Gmail 앱 비밀번호 (선택)

6.2 로컬용 Docker Compose 실행

```
cd echoforest-backend
docker-compose -f docker-compose.local.yml up -d --build
```

첫 실행 시 이미지 빌드로 약 5-10분 소요됩니다.

6.3 서비스 상태 확인

```
docker ps
```

정상 상태 예시:

NAME	STATUS
echoforest-server	Up
echoforest-ai	Up
echoforest-mysql	Up
echoforest-redis	Up
echoforest-livekit	Up

6.4 프론트엔드 실행 (별도 터미널)

```
cd echoforest-frontend
npm install
npm run dev
```

접속 URL: <http://localhost:5173>

6.5 로컬 접속 정보

서비스	URL	비고
프론트엔드	http://localhost:5173	Vite 개발 서버
백엔드 API	http://localhost:9001	Spring Boot
MySQL	localhost:3307	직접 접속 시
Redis	localhost:6379	직접 접속 시
LiveKit	ws://localhost:7880	WebSocket

6.6 서비스 중지

```
docker-compose -f docker-compose.local.yml down
```


[주의사항]

- 로컬 실행 시 Nginx, Jenkins, Prometheus, Grafana는 포함되지 않습니다.
- 프론트엔드는 Vite 개발 서버로 직접 실행합니다.
- AI 서버 경로가 다른 경우 .env에서 AI_REPO_PATH를 수정하세요.

7. 외부 서비스 정보

7.1 Gmail SMTP (이메일 발송)

용도: 게임 플레이 후 추억 이미지 이메일 전송

설정 방법:

1. Google 계정 > 보안 > 2단계 인증 활성화
2. Google 계정 > 보안 > 앱 비밀번호 생성
3. 생성된 16자리 비밀번호를 MAIL_PASSWORD에 입력

설정 값:

- 이메일: {MAIL_USERNAME}
- 앱 비밀번호: {MAIL_PASSWORD}

7.2 LiveKit (실시간 음성/영상)

용도: 4인 협동 플레이 시 실시간 음성 채팅

설정 (자체 호스팅):

- docker-compose.yml에 LiveKit 서버 포함
- LIVEKIT_API_KEY: {LIVEKIT_API_KEY}
- LIVEKIT_API_SECRET: {LIVEKIT_API_SECRET}

8. DB 덤프 파일

8.1 데이터베이스 백업 (덤프)

```
docker exec echoforest-mysql sh -c 'mysqldump -u root -p"$MYSQL_ROOT_PASSWORD" echoforest' > echoforest_dump.sql
```

8.2 데이터베이스 복원

```
docker exec -i echoforest-mysql sh -c 'mysql -u root -p"$MYSQL_ROOT_PASSWORD" echoforest'  
< echoforest_dump.sql
```

8.3 초기 스키마

Spring Boot JPA가 서버 시작 시 자동으로 테이블을 생성합니다.
(application.properties: spring.jpa.hibernate.ddl-auto=update)

=====

9. 시연 시나리오

=====

9.1 메인 페이지

URL: <https://i14d105.p.ssafy.io>

동작: 로그인/회원가입 버튼 클릭

9.2 회원가입 및 로그인

1. 회원가입 버튼 클릭
2. 아이디, 비밀번호, 이메일 입력
3. 회원가입 완료 후 로그인

9.3 로비 진입

1. 로그인 후 자동으로 로비 이동
2. 방 만들기 또는 방 코드 입력하여 참가

9.4 대기실

1. 4명이 되어야 게임 시작
2. 카메라/마이크 권한 허용
3. 모든 플레이어 Ready 시 게임 시작

9.5 인게임 (스테이지 1~4)

1. 4인 협동 퍼즐 게임 진행
2. 음성 인식: 부정어 사용 시 저주 스택 증가
3. 저주 스택 5 도달 시 랜덤 저주 발동
 - 거대화 저주: 캐릭터 크기 증가
 - 반전 저주: 좌우 조작 반전
 - 시한폭탄 저주: 제한 시간 내 해제 필요
4. 테스트/시연을 위해 p, q, n, spacebar를 동시에 누르면 엔딩미션으로 넘어감

9.6 엔딩 미션

1. 마지막 스테이지 클리어 후 포즈 따라하기 미션
2. MediaPipe를 이용한 모션 인식
3. 성공 시 스테이지 클리어

9.7 결과 화면

1. 게임 통계 표시 (클리어 시간, 저주 횟수 등)
2. 추억 이미지 저장
3. 이메일로 추억 이미지 전송 가능

=====

지원 및 문의

=====

팀 연락처: meahri.forest@gmail.com

=====

작성자: 이혜민

최종 수정일: 2026-02-08

=====